



Parallel Kernel Image Processing

Parallelizzazione e vettorizzazione di
filtri di convoluzione su immagini

Lorenzo Cappellini

Parallel Computing

Obiettivo

- Implementare **algoritmo di convoluzione** tra immagini e Kernel
- Applicare **tecniche di parallelizzazione e vettorizzazione** per velocizzare l'esecuzione
- **Verificare le prestazioni** delle tecniche utilizzate → misurare i tempi e valutare gli approcci in termini di ***speedup***

Kernel di convoluzione

- Formula:

$$O(x, y) = \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} I(x + i - r, y + j - r) K(i, j)$$

- Pixel di output calcolato come **combinazione pesata** dei valori del pixel in input
- Kernel** → matrice quadrata $K \times K$ con **coefficienti float**
- Filtri per immagini → *Gaussian blur*, *sharpening*, ***edge detection effect***
- Pixel indipendenti** → problema altamente parallelizzabile

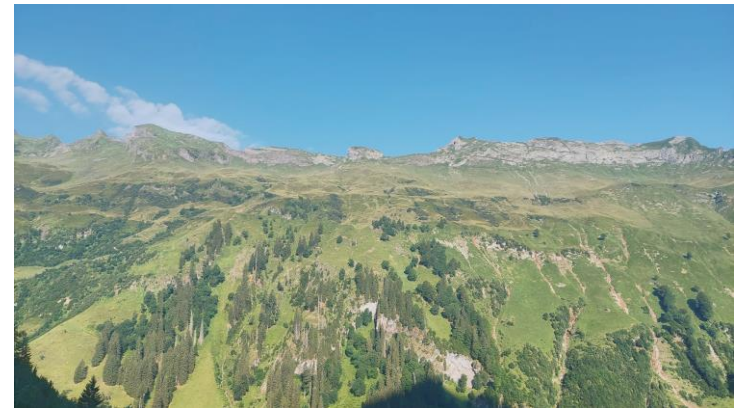


Immagine originale



Immagine dopo l'applicazione del *kernel* di *edge detection*

Flusso del programma

- Caricamento immagini (formato *.bmp*)
- Avvio algoritmo di **convoluzione** nella configurazione prestabilita
- Misurazione del **tempo impiegato** (*std::chrono::high_resolution_clock*)
- **Ripetizione** dell'operazione (e misurazione) più volte
- Salvataggio tempi in un file *.csv*
- Creazione **grafici** con script Python

Approcci confrontati

- **Base** → algoritmo sequenziale (layout AoS)
 - **OpenMP** → parallelismo multithread su CPU
 - **AVX2** → parallelismo con vettorizzazione SIMD
 - **OpenMP + AVX2** → combinazione delle due tecniche
 - **CUDA** → parallelismo su GPU
-
- **AoS vs SoA** → test aggiuntivo di confronto dei due layout di memorizzazione delle immagini

CPU: OpenMP e AVX2

OpenMP

- Ciclo sulle righe parallelizzato → **righe distribuite tra i thread**
- Direttiva `#pragma omp parallel for`
- **Pixel indipendenti** → no race conditions → **nessuna sincronizzazione** necessaria
- Creazione team e scheduling dei thread → introduce **overhead**

AVX2

- Sfrutta **registri da 256 bit** per eseguire una **stessa operazione su più dati** (SIMD)
- Vettorizzazione affidata al compilatore, specificando il flag `/arch:AVX2`
- Dipende dal **supporto della CPU**
- **Nessun overhead** richiesto

OpenMP+AVX2

- Combinazione delle due tecniche precedenti → **somma dei benefici** di entrambi

CUDA: GPU computing

- **Overhead** di allocazione (*cudaMalloc*) e trasferimento dati (*cudaMemcpy*)
- Implementato con schema a **tile** con **shared memory** → accessi molto più veloci
 - blocco di thread di dimensione $TILE_WIDTH \times TILE_WIDTH$
 - Gestione **pixel di bordo** → aggiunta **zona di contorno** (*halo*)
- **Convoluzione con 2-batch loading**
 - Area da caricare (*tile+halo*) maggiore del numero di thread del *tile* → **alcuni thread caricano 2 elementi**
 - **Primo caricamento**: ogni thread calcola l'**indice nell'area totale** partendo dal proprio indice nel blocco e **carica un pixel**
 - **Secondo caricamento**: gli stessi thread, **aggiunto un offset all'indice**, caricano un **secondo elemento** → solo se la destinazione ricade **dentro l'area**
- Pixel esterni ai limiti dell'immagine gestiti con *zero-padding*
- Immagine e kernel read-only → dichiarati **const __restrict__** → il compilatore può inserirle nella **cache costante** → accessi più veloci

AoS vs SoA

Array of Structures

- Pixel su singolo array: **RGBRGBRGB...**
- Layout più **comune e intuitivo** per immagini
- Usato per memorizzare l'immagine nel programma

Structure of Arrays

- Canali colore su 3 array separati: **RRR...** ; **GGG...** ; **BBB...**
- **Conversione da AoS a SoA esclusa dal benchmark**

- Confronto dei due layout a livello di **prestazioni**
- Convoluzione applicata indipendentemente sui 3 canali colore → **nessuna reale differenza** nella versione sequenziale base
- Applicata **vettorizzazione AVX2** → efficienza maggiore per operazioni su dati salvati su **memoria contigua** → idealmente **SoA più efficiente**

Setup dei benchmark

Dataset e misurazioni

- Immagini 16:9 con **dimensioni standard**
- **50 misurazioni** per immagine (per ogni configurazione)
- **Mediana** dei valori → maggiore **resistenza agli outliers**

Hardware utilizzato

- **CPU** → *AMD Ryzen 7 3700X 8-Core*
- **RAM** → 16 GB DDR4
- **GPU** → *NVIDIA RTX 3060 Ti*
- Sistema operativo → Windows 10

Nome	Risoluzione	Numero di pixel
nHD	640 × 360	230.400
qHD	960 × 540	518.400
HD	1280 × 720	921.600
WXGA	1366 × 768	1.049.088
HD+	1600 × 900	1.440.000
Full HD	1920 × 1080	2.073.600
QHD	2560 × 1440	3.686.400
4K UHD	3840 × 2160	8.294.400
5K	5120 × 2880	14.745.600
8K UHD	7680 × 4320	33.177.600
16K UHD	15360 × 8640	132.710.400

Tabella con risoluzioni delle immagini
utilizzate nei test

Metrica: speedup

- Formula speedup: $S = \frac{T_{seq}}{T_{par}}$
- $T_{seq} \rightarrow$ **tempo** della versione **Base** (sequenziale)
- $T_{par} \rightarrow$ **tempo** della **configurazione** in analisi
- Confronto a parità di dimensione di immagine
- Misurazione ripetuta **50 volte** per ogni configurazione
- **Mediana** dei valori ottenuti $\rightarrow$ meno influenza del rumore
- Tempi di esecuzione molto brevi $\rightarrow$ *millisecondi* non sufficienti $\rightarrow$ misurazione effettuata in *microsecondi* $\rightarrow$ convertiti in ms per i grafici



Risultati: AoS vs SoA

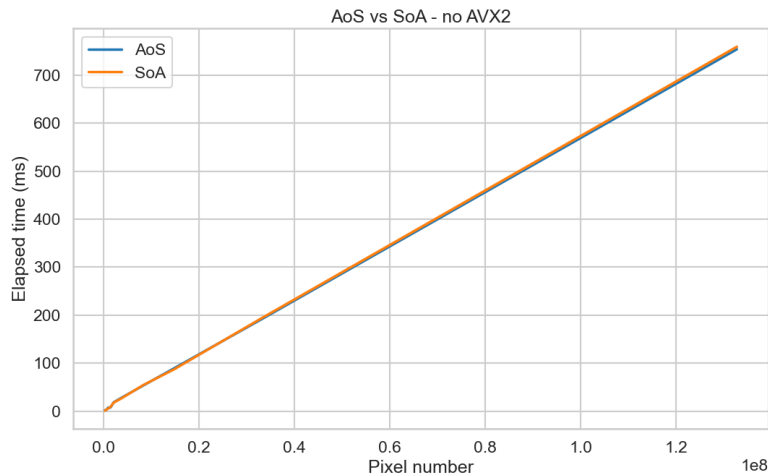


Grafico tempo impiegato con i due layout SENZA AVX2

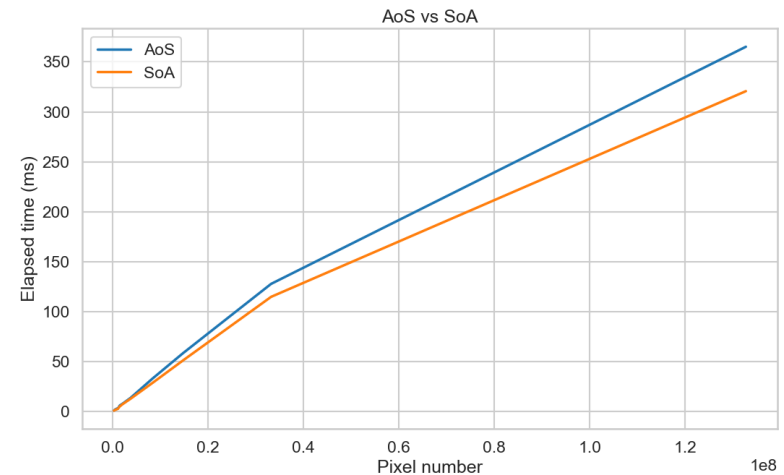


Grafico tempo impiegato con i due layout CON AVX2

- **No vettorizzazione** → **nessuna differenza** tra i due layout
- **Vettorizzazione** più efficiente in presenza di operazioni su **dati contigui** in memoria → AVX2 sfrutta meglio il **layout SoA** → **prestazioni leggermente migliori**

Tempi di esecuzione

- Pixel aumentano → **tempo aumenta linearmente** → **complessità $O(N)$**
- **Base** → più lenta
- **OpenMP** più veloce di **AVX2**
- **OpenMP + AVX2** → combina i due livelli ottenendo ottime prestazioni
- **CUDA** → **prestazioni migliori in assoluto** con carichi grandi

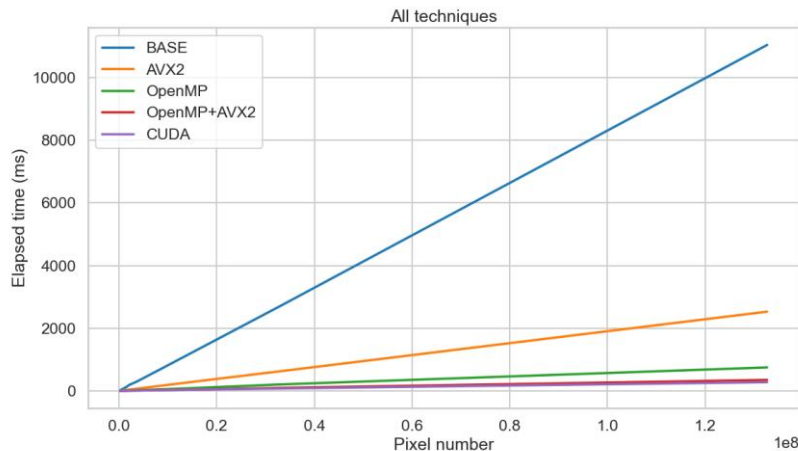


Grafico dei tempi impiegati da ogni configurazione

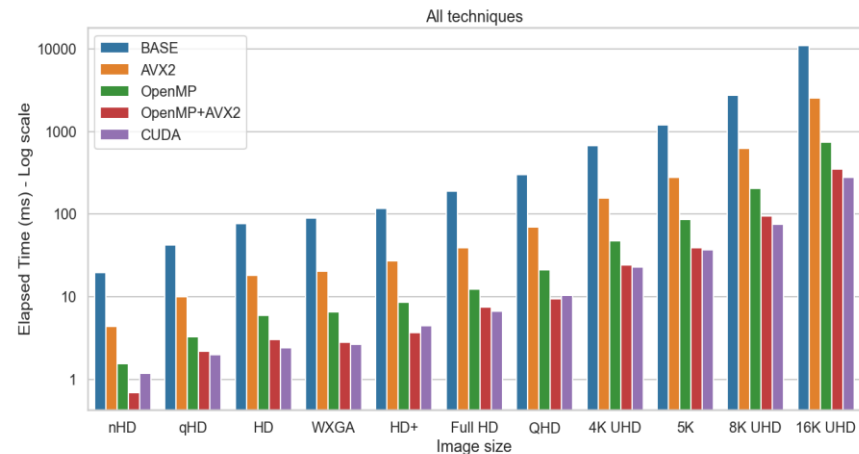
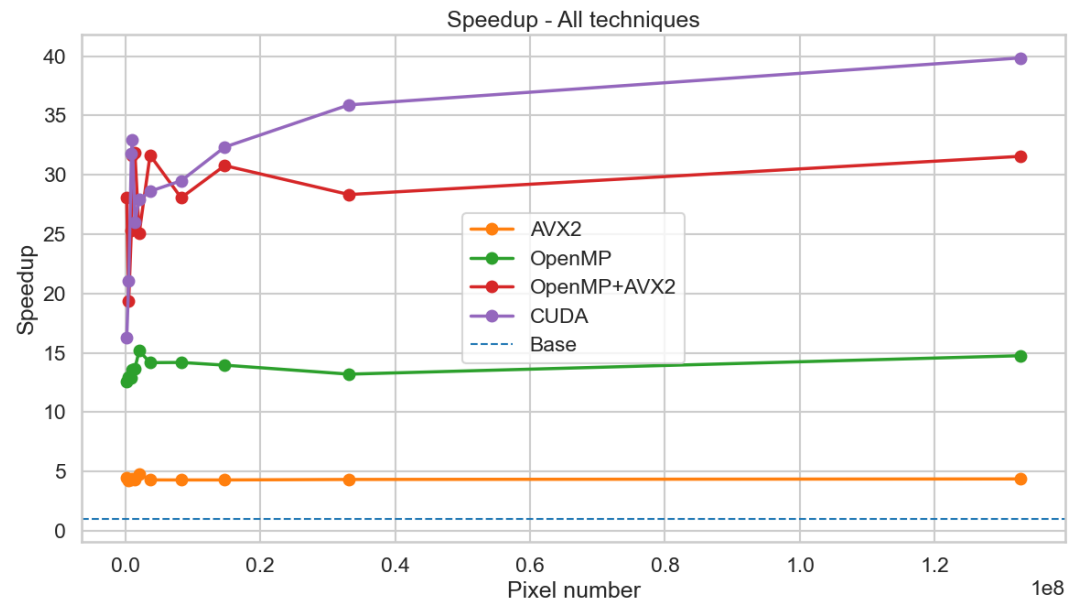


Grafico a barre dei tempi impiegati da ogni configurazione
(scala logaritmica)

Speedup



- **AVX2** → speedup praticamente **costante** → stesso numero di operazioni per istruzione
- **OpenMP** → speedup **cresce** → **overhead** iniziale → pesa meno con immagini grandi
- **CUDA** → andamento **crescente** → costo pari a CPU con pochi pixel → **overhead** di allocazione e copia dati notevole
- **CUDA** → **migliore** in assoluto per **immagini grandi**
- **Oscillazioni** con pochi pixel → tempi di esecuzione molto brevi → **rumore di misurazione** molto impattante
- Speedup → rapporto tra due tempi → piccoli errori causano grandi variazioni percentuali

Conclusioni

- **Risoluzione** immagini tendenzialmente sempre **in aumento** nella storia
- **CUDA** è la soluzione **più efficace** per **immagini grandi** → richiesto **hardware aggiuntivo** (GPU NVIDIA)
- **CPU**: parallelismo di OpenMP e vettorizzazione AVX2 combinati offrono un'**alternativa** comunque **molto valida**
- **AVX2**: migliora le prestazioni ed è **utile** quando **multithreading non disponibile**





Parallel Kernel Image Processing

Parallelizzazione e vettorizzazione di
filtri di convoluzione su immagini

Lorenzo Cappellini

Parallel Computing